Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas

# Título del Práctica:

# Integración de Syscalls: Sistema de Monitoreo y Seguridad con Dashboard en Tiempo Real

**PONDERACIÓN: 10 pts**

**Tiempo estimado: 10 hrs**

Índice

**1. MARCO FORMATIVO..... 3**
1.1. Valor..... 3
1.2. Competencia(s)..... 3
1.3. Objetivo SMART..... 3
**2. Enunciado de la Práctica..... 4**
3.1 Descripción del problema a resolver..... 4
3.2 Alcance de la práctica..... 4
3.3 Entregables..... 4
3. Material de apoyo..... 6
**6. Recursos y herramientas a utilizar..... 7**
**7. Cronograma..... 7**
**8. Rúbrica de Calificación..... 7**
8.1 Requisitos para optar a la calificación..... 7
**8.2 Resumen de Puntuaciones..... 9**
8.5 Comentarios Generales..... 9
Detalle de la Calificación..... 11

# 1. MARCO FORMATIVO

## 1.1. Valor

| Nombre del valor | ¿Cómo se aplica en tu laboratorio? |
|---|---|
| Criterio y toma de decisiones | El estudiante debe decidir cómo orquestar múltiples mecanismos del kernel construidos en prácticas anteriores (consulta de procesos, comunicación entre procesos, monitoreo de eventos) para que trabajen juntos de forma coherente, priorizando estabilidad y claridad arquitectónica sobre complejidad innecesaria. |

## 1.2. Competencia(s)

| Tipo de Competencia | |
|---|---|
| Competencia General | Actualiza permanentemente sus conocimientos relacionados con TIC en general, apoyándose en las estrategias de aprendizaje apropiadas. |
| Competencia Específica | Identifica oportunidades y riesgos para la innovación y adaptación de conocimientos y tecnologías para resolver problemas. |

## 1.3. Objetivo SMART

| **Específico (¿Qué?)** | **Medible (¿Cuánto?)** | **Alcanzable (¿Cómo?)** | **Realista (¿Para qué?)** | **A Tiempo (¿Cuándo?)** |
|---|---|---|---|---|
| Integrar las syscalls desarrolladas en las prácticas 2 y 3 (sys_get_process_info, sys_get_processes_state, sys_ipc_channel_send/receive, sys_start_log_watch/stop_log_watch) en un sistema de monitoreo con backend y dashboard web en tiempo real. | El sistema se considera exitoso si el dashboard muestra correctamente las métricas de memoria, la tabla de procesos activos, y un panel de eventos de seguridad alimentado por el mecanismo de log_watch, todo actualizándose sin intervención manual. | El estudiante reutilizará el código de sus syscalls previas, implementará la nueva syscall de memoria, y desarrollará un programa intermedio en C y un dashboard web que consuma esa información. | Para que el estudiante comprenda de manera tangible la transición entre el espacio de usuario y el espacio de kernel, así como el impacto directo de la programación de sistemas en el comportamiento del hardware. | Para que el estudiante experimente el paso de "mecanismos aislados" a "sistema integrado", ejercitando la misma lógica de integración que exigirá el proyecto final. |

# 2. Enunciado de la Práctica

En esta práctica el estudiante integrará el trabajo realizado en las Prácticas 2 y 3, reutilizando las syscalls previamente desarrolladas para construir un sistema completo de monitoreo del sistema operativo con visualización en tiempo real. El objetivo es demostrar cómo mecanismos que se construyeron de forma aislada (consulta de procesos, comunicación entre procesos, monitoreo de eventos) pueden combinarse para formar una herramienta coherente de instrumentación — el mismo tipo de integración que se espera en el proyecto final del curso. Para lograrlo, el estudiante deberá:

- Reutilizar `sys_get_process_info` / `sys_get_processes_state` para poblar una tabla de procesos consultable.
- Rediseñar el uso del canal IPC (`sys_ipc_channel_send`/`receive`) para que el daemon de monitoreo reaccione a eventos en vez de hacer polling constante.
- Reutilizar `sys_start_log_watch`/`stop_log_watch` para alimentar un panel de "eventos de seguridad" en el dashboard.
- Construir un programa intermedio en C que orqueste las syscalls anteriores, lea métricas de memoria desde `/proc/meminfo`, y transforme toda la información a JSON.
- Construir un dashboard web que visualice todo en tiempo real.

## 2.1. Descripción del problema a resolver

Las herramientas de monitoreo e instrumentación reales (como los agentes de seguridad EDR o los dashboards de observabilidad) no dependen de un solo mecanismo del kernel — combinan múltiples fuentes de información (estado de procesos, eventos del sistema, métricas de recursos) y las presentan de forma unificada para que un analista pueda tomar decisiones.

Hasta ahora, cada práctica construyó una pieza aislada:

- Práctica 2: consulta de información de procesos bajo demanda.
- Práctica 3: comunicación entre procesos y monitoreo autónomo de eventos.

En esta práctica, el estudiante deberá resolver el problema de integrar estas piezas en un sistema unificado, demostrando que comprende no solo cómo construir cada mecanismo por separado, sino cómo coordinarlos entre sí.

## 2.2. Alcance de la práctica

El sistema estará compuesto por tres capas:

- **1. Kernel Modificado (Máquina Virtual):** contiene las syscalls ya desarrolladas en P2 y P3. No se requiere implementar ninguna syscall nueva.
- **2. Backend / Programa Intermedio (Máquina Virtual):** actúa como orquestador. Invoca las syscalls, lee `/proc/meminfo` para las métricas de memoria, transforma la información a JSON, y la expone vía HTTP/WebSocket.

- **3. Dashboard Web (puede ejecutarse en el host):** interfaz visual que consume la información del backend y la muestra en tiempo real.

### 2.2.1 Syscalls a integrar (todas ya desarrolladas — no hay syscalls nuevas)

- `sys_get_process_info(pid)` — información detallada de un proceso.
- `sys_get_processes_state()` — estado de todos los procesos activos.
- `sys_ipc_channel_send(mensaje)` / `sys_ipc_channel_receive()` — canal de comunicación entre procesos.
- `sys_start_log_watch(rutas, log_central, palabra_clave)` / `sys_stop_log_watch(id)` — monitoreo de eventos vía hilo de kernel.

### 2.2.2 Rol de cada mecanismo dentro del sistema

Cada syscall reutilizada debe tener una función clara dentro del dashboard, no simplemente estar presente:

- `sys_get_process_info`/`sys_get_processes_state`: alimentan la tabla de procesos activos del dashboard, con la posibilidad de consultar el detalle de un proceso específico por PID.
- `sys_ipc_channel_send`/`receive` — uso rediseñado: el daemon no debe hacer polling constante revisando si hay eventos de seguridad. En su lugar, debe existir un mecanismo (puede ser el propio hilo de `log_watch` extendido, o lógica adicional) que envíe un mensaje por el canal IPC cuando `log_watch` detecte la palabra clave en algún archivo. El daemon en espacio de usuario debe quedar bloqueado en `sys_ipc_channel_receive` esperando esas notificaciones, y solo entonces actualizar el panel de eventos del dashboard. Esto demuestra un patrón productor-consumidor real entre kernel y espacio de usuario, evitando desperdiciar CPU revisando constantemente si hay novedades.
- `sys_start_log_watch`/`stop_log_watch`: siguen encargándose de vigilar los archivos de log; ahora, además, disparan la notificación IPC descrita arriba cuando detectan una coincidencia.

Métricas de memoria (`/proc/meminfo`): el daemon lee este archivo (interfaz ya expuesta por el kernel, sin necesidad de código de kernel adicional) para poblar las gráficas de memoria y páginas del dashboard.

### 2.2.3 Programa intermedio (Daemon en C)

El programa intermedio debe:

- Iniciar el monitoreo de eventos (`log_watch`) y quedar a la espera de notificaciones vía el canal IPC, en vez de hacer polling continuo del panel de seguridad.

- Invocar `sys_get_processes_state` de forma periódica (esto sí puede ser polling, ya que es una consulta de estado, no un evento) para mantener la tabla de procesos actualizada.
- Leer `/proc/meminfo` periódicamente para las métricas de memoria.
- Estructurar toda la información en JSON.
- Exponer los datos al dashboard vía HTTP o WebSocket.
- Ejecutarse de forma continua, como un servicio/daemon.
- Manejar errores de forma adecuada (ej. si una syscall falla, no debe caerse todo el daemon).

El JSON generado debe incluir al menos:

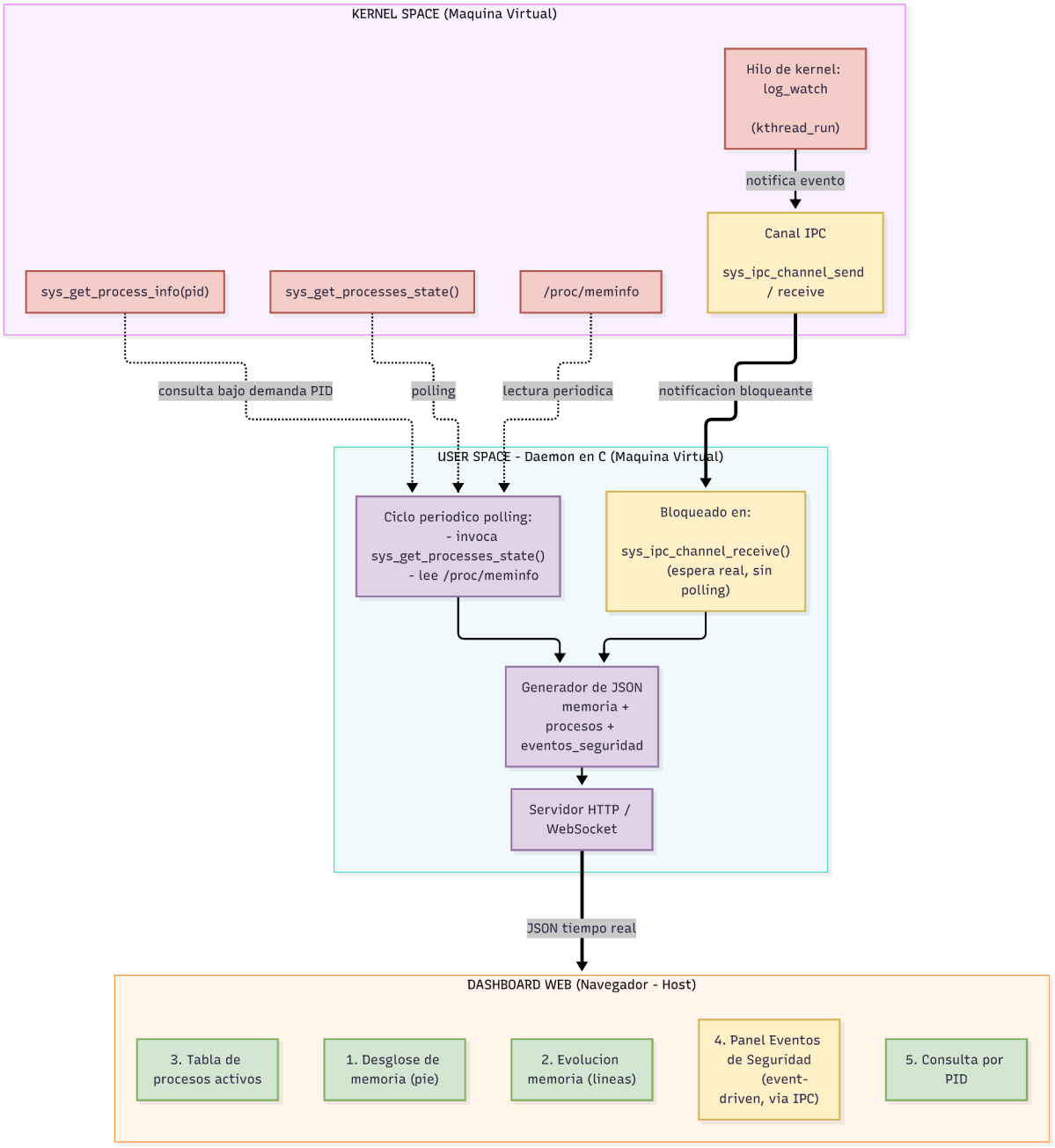
```json
{
  "memoria_usada": 0,
  "memoria_libre": 0,
  "memoria_cache": 0,
  "swap_usada": 0,
  "procesos": [
    {"nombre": "", "pid": 0, "estado": ""}
  ],
  "eventos_seguridad": [
    {"archivo_origen": "", "palabra_detectada": "", "contenido": "", "timestamp": ""}
  ]
}
```

### 2.2.4 Dashboard web (Frontend)

Debe incluir, como mínimo, estas visualizaciones:

- Desglose de uso de memoria — gráfico de pie (memoria usada, libre, caché), a partir de `/proc/meminfo`.
- Evolución del uso de memoria en el tiempo — gráfico de líneas (memoria física vs. swap).
- Tabla de procesos activos — con nombre, PID y estado, alimentada por `sys_get_processes_state`.
- Panel de Eventos de Seguridad — tabla en tiempo real alimentada por el mecanismo de notificación IPC descrito en 2.2.2, mostrando los hallazgos más recientes.
- Campo de consulta por PID — al ingresar un PID, el dashboard debe consultar `sys_get_process_info` y mostrar el detalle del proceso.

El dashboard debe actualizarse en tiempo real (intervalos regulares o mediante WebSocket), y el panel de eventos de seguridad en particular debe reflejar el patrón event-driven descrito en 2.2.2 (no debe depender de refrescar la página ni de polling ciego).




## 2.3 Requerimientos técnicos

Kernel Linux: Versión 6.12.69 Long Term(LT).

Lenguaje C: para la implementación de la syscall y el programa de usuario.

Linux Mint u otra distribución basada en Debian.

Editor de texto: VS Code, Vim, Nano.

Terminal de Linux: para compilación, pruebas y depuración.

Opcional: `gdb`, `strace`, `dmesg` para depuración avanzada.

# 3. Entregables

| Tipo | Descripción |
|---|---|
| **Repositorio en GitLab** | Enlace a la rama (Practica_4_2S2026, nombre = carné) con el código del kernel modificado, todas las syscalls (reutilizadas), el programa intermedio y el dashboard. Debe incluir instrucciones claras para compilar y ejecutar todo el sistema. |
| **Ajustes a Syscalls existentes** | Evidencia de la correcta reutilización/adaptación de las syscalls de P2 y P3, especialmente el rediseño del canal IPC como mecanismo de notificación desde log_watch. |
| **Programa de usuario (Daemon)** | Código en C del daemon, incluyendo el ejecutable compilado, que orquesta las syscalls, lee /proc/meminfo y genera el JSON. |
| **Dashboard Web** | Código del dashboard con las 5 visualizaciones requeridas. |
| **Manual Técnico** | Documento en Markdown que describa: arquitectura general del sistema, qué rol cumple cada syscall, el diseño del mecanismo de notificación vía IPC (por qué se diseñó así y no con polling puro), y al menos 3 errores encontrados durante la integración con su solución. |

# 4. Material de apoyo


- Love, R., & Bovet, D. (2005). *Understanding the Linux Kernel*. O'Reilly Media.
- Kerrisk, M. (2010). *The Linux Programming Interface*. No Starch Press.
- Documentación oficial del kernel Linux: https://www.kernel.org/doc/html/latest/


# 5. Recursos y herramientas a utilizar

**Software:** GCC, Make, Git, editor de texto, `gdb`, `strace`.

**Plataformas:** GitLab(repositorio), UEDI o Classroom (entrega).

**Lecturas recomendadas:** Documentación de kernel Linux, manuales de programación en C, artículos sobre llamadas al sistema y manejo de procesos.

# 6. Cronograma

El cronograma describe las etapas clave de la práctica, los plazos estimados para cada una, y el proceso de asignación, elaboración y calificación de las tareas. Los estudiantes deberán seguir este plan para asegurar que el práctica avance de manera organizada y cumpla con los plazos establecidos. Cada fase incluye la asignación de tareas, el tiempo estimado para su elaboración, y el momento de su calificación.

| Tipo | Fecha Inicio | Fecha Fin |
|---|---|---|
| Asignación de Práctica | 18/09/2026 | 18/09/2026 |
| Elaboración | 18/09/2026 | 06/10/2026 |
| Calificación | 07/10/2026 | 09/10/2026 |

# 7. Rúbrica de Calificación

7.1 Requisitos para optar a la calificación

| Tema | Descripción | Cumple (Sí/No) |
|---|---|---|
| Cumplimiento de la tecnología establecida | La práctica debe desarrollarse en un sistema GNU/Linux basado en Debian, en VM o instalación nativa. | |
| Integridad de la entrega | Se debe entregar el enlace a la rama del repositorio GitLab con el código completo y la documentación. | |
| Compilación Funcional | El kernel modificado debe arrancar (boot) sin errores. | |
| Kernel modificado | Debe utilizarse el mismo kernel compilado desde la Práctica 1. | |
| Sin polling puro en eventos de seguridad | El panel de "Eventos de Seguridad" debe alimentarse vía el mecanismo de notificación IPC descrito en 2.2.2, no mediante refresco ciego constante. | |

7.2 Resumen de Puntuaciones

| Área | Puntos Totales | Puntos Obtenidos |
|---|---|---|
| Habilidades | 70 | |
| Correcta reutilización de `sys_get_process_info`/`sys_get_processes_state` | 15 | |
| Rediseño del canal IPC como mecanismo de notificación (sin polling) | 25 | |
| Programa intermedio (daemon) funcional, incluyendo lectura de `/proc/meminfo` | 15 | |
| Dashboard funcional con las 5 visualizaciones requeridas | 10 | |
| Integración completa del sistema (kernel + daemon + dashboard) | 5 | |
| Conocimientos | 30 | |
| Pregunta 1 | 5 | |
| Pregunta 2 | 5 | |
| Documentación técnica (Manual Técnico) | 20 | |
| **TOTAL** | **100** | |

## Detalle de la Calificación

| No. | Criterio | Punteo Máx. | Satisfactorio (100-61%) | Nece |
|---|---|---|---|---|
| 1 | Reutilización de `sys_get_process_info`/`sys_get_processes_state` | 15 | Ambas syscalls funcionan correctamente dentro del sistema integrado y alimentan la tabla de procesos. | Alguna syscall no integrada. |
| 2 | Canal IPC como notificación | 25 | El daemon se bloquea esperando eventos reales desde `log_watch` (no polling ciego) y reacciona correctamente al detectar una coincidencia. | El daemon usa pol mecanismo no blo |
| 3 | Daemon funcional | 15 | Orquesta todas las syscalls, lee `/proc/meminfo` correctamente, genera JSON completo, maneja errores. | Falla al invocar sys memoria correctam |
| 4 | Dashboard funcional | 10 | Las 5 visualizaciones funcionan y se actualizan correctamente. | Faltan visualizacio correctamente. |
| 5 | Integración completa | 5 | El sistema completo funciona de punta a punta sin intervención manual. | Requiere pasos m capas. |
| 6 | Pregunta 1 (arquitectura) | 5 | Explica correctamente cómo se integran las syscalls y por qué. | Explicación incomp |
| 7 | Pregunta 2 (IPC/notificación) | 5 | Explica correctamente las ventajas del mecanismo de notificación sobre polling puro. | Explicación incomp |
| 8 | Documentación técnica | 20 | Completa, clara, explica arquitectura, roles de cada syscall y decisiones de diseño. | Incompleta, desorg razonamiento de in |

## Comentarios Generales

_____

_____

_____

_____

_____

_____

_____